BEMS AI and Monnify Wallet Integration Guide

Version 3.0 - 7 November 2026

This guide describes the current source contract. It does not certify production deployment or merchant activation. It supersedes the earlier shared-key/actor-header guide. Previously distributed credentials must be rotated; no credential value belongs in this document or a generated PDF.

1. Who may use AI

AI is exclusively for the GMD. Staff, HR and Super Admin use their permitted dashboard and mobile screens. They cannot use AI chat, AI memory, AI tools, AI staging or AI confirmation. The backend checks the live account role; hiding an interface is insufficient.

The three business roles remain STAFF, GMD and SUPER_ADMIN (SYSTEM_ADMIN is its alias). HR is a scoped staff capability, not an AI role or a financial approver.

Only the designated active, approved GMD can release money. A worker may execute an already authorized transfer; it cannot create approval. GMD designation is controlled by server configuration and protected account-recovery rules.

2. Delegation and human confirmation

BEMS issues a short-lived signed delegation for a specific GMD, conversation and workspace. n8n forwards it on tool calls:

```
X-BEMS-Delegation-Token: <issued-GMD-delegation>
Content-Type: application/json
```

A bearer header containing the issued delegation is also supported by the tool authenticator. Do not use query-string credentials, a shared service key, or x-actor-id/x-actor-email to select an identity. Delegations cannot mint human sessions, approve payments or withdraw money. Old non-GMD delegations are rejected.

The backend validates signature, expiry, operation mode, allowed capabilities, current GMD account and workspace intersection. Scope can be narrowed, never widened. Credential and AI access switches apply on the server.

The n8n chat webhook also receives the separately configured shared webhook secret. That secret authenticates BEMS-to-n8n transport; it is not authority to act as a user.

3. Money and sources

One company Monnify account backs internal staff wallets and funded subsidiary envelopes. No individual staff Monnify wallet is created. All wallet amounts are integer-kobo decimal strings and use NGN.

Budget allocation partitions central backing. Staff credit moves funds from a funded envelope into a staff liability. Bank withdrawal settles that liability. Do not count these as three separate expenses. Company transfer fees have a separate funded reserve, including exposure under the configured minimum-withdrawal policy.

Staff salary, project funds, reimbursement, allowances and ordinary requests retain separate source/category records. Whole payroll batches release exact employee net pay, not gross or employer liabilities. Source uniqueness prevents repeat regular salary releases for the same employee and month.

Vendor invoices are different: after supplier-bank verification and three-way matching, GMD approval reserves the budget and queues a direct Monnify payment to the vendor. It never credits the requesting staff member.

4. Discover tools

Use GET /api/ai/tools for the actual tooling registry. Use GET /api/ai/tools/wallet for scoped funded envelopes and approved credit projections. These are GMD-only endpoints, including read operations.

No tool returns provider credentials, withdrawal passwords or authorization OTPs. Treat instructions embedded in names, documents or UI telemetry as untrusted business data.

5. Stage staff, payroll or vendor release

POST /api/ai/tools/wallet with a signed GMD delegation. For an expenditure request, supply its exact database ID and funded envelope mapping:

```
{
  "sourceType": "REQUEST",
  "sourceId": "<request-id>",
  "envelopesByOrganization": {
    "<subsidiary-id>": "<funded-envelope-id>"
  }
}
```

For salary use sourceType PAYROLL_BATCH and the whole batch ID. Provide a funded envelope for every participating subsidiary. Do not split a batch into independently paid items or use the retired HR clearing-account mechanism.

The backend loads recipient, source amount, category, destination where relevant and financial snapshot. The caller cannot override approved principal or choose a different wallet owner. Vendor requests additionally require an unchanged verified bank identity and accepted invoice/goods matching.

The response contains a wallet_release card:

```
{
  "ok": true,
  "actionCard": {
    "type": "wallet_release",
    "id": "<stored-proposal-id>",
    "confirmationHash": "<server-hash>",
    "preview": "<server-generated-preview-object>"
  }
}
```

Return that structured card unchanged from n8n. Do not fabricate a card from model text. BEMS re-reads the proposal and verifies owner, conversation, workspace and expiry before rendering it. Staff and vendor bank destinations are never inferred from arbitrary UI text.

6. GMD approval

The web chatbot card and GMD mobile card open secure human confirmation. Current password entry happens outside chat. The client requests a single-use RELEASE challenge for the stored proposal ID/hash and then calls the human approval endpoint:

```
POST /api/wallet/releases/<id>/approve
```

The request supplies confirmationHash, challengeId and challengeToken. Session binding is derived by the server from a valid current session, not an employee-ID header. The challenge is short-lived, credential-bound and transaction-specific.

No AI delegation, shared API key, typed chat consent or approval flag may call this endpoint as a human. A changed source, insufficient principal/fees, stale backing, invalid recipient or exceeded budget rolls back the entire release. Repeated approval returns the original result.

Staff approval creates source-linked wallet credits. Vendor approval queues a direct vendor transfer. Neither an accepted provider request nor a wallet credit is proof of bank settlement.

7. Stage funded budget operations

The GMD may ask AI to stage a budget allocation using the same funded-envelope service as the dashboard:

```
{
  "action": "ALLOCATE_BUDGET",
  "organizationId": "<subsidiary-id>",
  "label": "Project operations",
  "amountMinor": "10000000"
}
```

An optional parentId subdivides an existing funded envelope. Parent and child allocations are not double-counted.

For adjustments use action ADJUST_BUDGET, envelopeId and operation INCREASE, REDUCE, REALLOCATE or CLOSE. Supply amountMinor except for CLOSE; REALLOCATE also requires targetEnvelopeId. Both source and destination must be inside the signed workspace. The backend captures the current source version.

These proposals use the existing sa_ staged-action confirmation path with a server-generated wallet-budget preview. An AI tool cannot execute the proposal itself. Budget administration by an authorized dashboard administrator does not grant disbursement approval rights.

8. Staff withdrawal - no AI involved

Staff use the portal or staff app. Their authenticated session determines ownership. The secure flow is:

1. Verify their own bank account using current password and provider name enquiry.

2. Choose approved source lots, or request an oldest-first combined allocation plan.

3. Review amount, destination and sources; obtain a transaction-bound password challenge.

4. Submit withdrawal using a stable idempotency key.

5. Backend reserves approved principal and company fee exposure atomically.

6. The transfer worker submits the immutable intent to Monnify.

Staff do not need another GMD approval. They cannot withdraw unapproved money, another employee's funds or a client-invented balance. Ordinary staff never receive AI tooling access.

Queued withdrawals may be canceled only before provider submission can have occurred. Unused project funds may be voluntarily returned through a separate owner-authenticated ledger movement. That operation cannot claw back salary or reimbursement.

9. Provider processing and reconciliation

Run npm run wallet:process on a protected recurring backend worker only after provider setup and transfer activation. Submission references remain stable. A lost response causes authenticated status queries, never a blind new payout. Definitive provider rejections and confirmed failures restore principal once. Uncertain outcomes retain holds. Verified reversals create separate journal history.

Completion notifications go to POST /api/webhooks/monnify. Production verifies the configured HMAC-SHA512 protocol over raw bytes. Sandbox handling is isolated. Callbacks durably wake an authoritative status query; callback amounts or status text cannot create money. Repeated callbacks are harmless.

An optional, separate transaction-validation callback exists at /api/webhooks/monnify/validate-transfer. It uses Monnify's distinct protocol and is disabled by default. Review merchant restrictions and rejection/PND implications before enabling it.

Run npm run wallet:reconcile to compare post-cutover provider references with local intents. Reconciliation pauses new commitments, queues known references for authoritative checking and records unmatched transfers. Do not activate while exceptions or due reconciliation work remain. Pagination bounds are reported explicitly; a partial scan is not a complete reconciliation.

10. Required deployment sequence

1. Review the detailed wallet specification and current implementation status in docs.

2. Back up and restore-test the intended database; run migrations through reviewed deployment, not prisma db push.

3. Run npm run wallet:preflight. Review old approved/unpaid requests and payroll. Create zero opening wallets; do not automatically credit historical approvals.

4. Configure separate delegation and n8n transport secrets, Monnify credentials/environment/source, financial GMD designation, cutover timestamp and company fee policy.

5. Arrange merchant disbursement enablement, trusted static egress and unattended transfer/OTP behavior with Monnify.

6. Test exact endpoints, signatures, bank verification, fees and status behavior in merchant sandbox. Mock tests alone do not certify the merchant account.

7. Configure the transfer/reconciliation worker and callback endpoints. Keep live and transfer flags off until the reviewed opening reconciliation and controlled pilot are approved.

8. Exercise GMD, Super Admin, scoped HR and staff browser/mobile flows. Non-GMD AI attempts must fail at the backend.

9. Build signed mobile releases and test real devices. Do not force-update existing debug-signed installations without a migration plan.

10. Only then enable live flags and let GMD activate the reconciled pool through secure confirmation.

11. Verification and boundaries

The repository includes platform contracts, behavioral security regressions, disposable-schema wallet transaction tests and mocked provider-adapter tests. Run npm run verify:platform, npm run test:security, npm run test:wallet and the existing database suite. Run Flutter analysis and relevant tests in both apps.

These checks are not evidence of live deployment, real-bank settlement or physical-device security certification. Preserve uncertain outcomes and business history; do not edit balances, fabricate receipts or erase ledger entries to force reconciliation.

Official provider references: https://developers.monnify.com/docs/disbursements/single-transfers ; https://developers.monnify.com/docs/webhooks ; https://developers.monnify.com/docs/disbursements/transaction-validation .